

测试方案

文档版本：v1.0

编写日期：2026 年 3 月 14 日

开发负责人：王舟

一、测试策略

1.1 测试层级

层级	测试类型	工具	覆盖率目标
单元测试	函数/方法级别	cargo test	>80%
集成测试	模块间交互	cargo test + httpx	核心流程 100%
E2E 测试	端到端流程	Playwright	关键路径覆盖
性能测试	负载/压力	wrk / locust	QPS>500
安全测试	漏洞扫描	cargo-audit	无高危漏洞

1.2 测试环境

环境	用途	数据
开发环境	开发人员本地测试	模拟数据
测试环境	CI/CD 自动化测试	脱敏数据
预发布环境	上线前验证	生产数据副本
生产环境	线上监控	真实数据

1.3 测试流程

开发提交代码

↓

CI 触发测试

↓

1. 代码检查 (clippy/fmt)	
2. 单元测试	
3. 集成测试	
4. E2E 测试 (关键路径)	
5. 代码覆盖率检查	

↓
全部通过 → 合并代码
↓
部署到预发布环境
↓
人工验证 → 生产发布

二、单元测试

2.1 测试框架

```
# Cargo.toml
[dev-dependencies]
tokio-test = "0.4"
fake = { version = "2.9", features = ["derive"] }
assert-json-diff = "2.0"
```

2.2 测试示例

```
// src/services/case_service_test.rs
#[cfg(test)]
mod tests {
    use super::*;
    use crate::test_utils::setup_test_db;
    use fake::faker::name::raw::Name;
    use fake::Fake;

    /// 测试创建案件
    #[tokio::test]
    async fn test_create_case() {
        // 准备测试数据
        let db = setup_test_db().await;
        let service = CaseService::new(db);
        let user_id = "user_test_001";

        let input = CreateCaseInput {
            name: Name().fake::<String>(),
            description: Some("测试案件".to_string()),
            tags: vec!["测试".to_string()],
        };
    }
}
```

```

};

// 执行测试
let case = service.create_case(user_id, &input).await.unwrap();

// 验证结果
assert_eq!(case.name, input.name);
assert_eq!(case.created_by, user_id);
assert!(case.id.starts_with("case_"));

// 验证数据库记录
let found = service.db.get_case(&case.id).await.unwrap();
assert!(found.is_some());
}

/// 测试案件名称重复
#[tokio::test]
async fn test_create_case_duplicate_name() {
    let db = setup_test_db().await;
    let service = CaseService::new(db);
    let user_id = "user_test_001";

    let input = CreateCaseInput {
        name: "重复案件".to_string(),
        description: None,
        tags: vec![],
    };

    // 创建第一个案件
    service.create_case(user_id, &input).await.unwrap();

    // 尝试创建同名案件，应该失败
    let result = service.create_case(user_id, &input).await;
    assert!(result.is_err());
    assert_eq!(
        result.unwrap_err().to_string(),
        "案件名称已存在"
    );
}

```

```

/// 测试获取案件列表
#[tokio::test]
async fn test_get_case_list() {
    let db = setup_test_db().await;
    let service = CaseService::new(db);
    let user_id = "user_test_001";

    // 创建测试数据
    for i in 0..10 {
        service.create_case(user_id, &CreateCaseInput {
            name: format!("案件{}", i),
            description: None,
            tags: vec![],
        }).await.unwrap();
    }

    // 获取列表
    let (cases, total) = service
        .get_case_list(user_id, 1, 20)
        .await
        .unwrap();

    assert_eq!(total, 10);
    assert_eq!(cases.len(), 10);
}
}

```

2.3 测试工具函数

```

// src/test_utils.rs
use sqlx::{SqlitePool, sqlite::SqlitePoolOptions};
use crate::AppState;

/// 设置测试数据库
pub async fn setup_test_db() -> SqlitePool {
    // 使用内存数据库
    let pool = SqlitePoolOptions::new()
        .max_connections(5)
        .connect("sqlite::memory:")
        .await
}

```

```

        .unwrap();

// 运行数据库迁移
sqlx::query(include_str!("../migrations/001_init.sql"))
    .execute(&pool)
    .await
    .unwrap();

    pool
}

/// 创建测试应用状态
pub fn create_test_state(pool: SqlitePool) -> AppState {
    AppState {
        db: Database::new(pool),
        auth_service: AuthService::new(pool.clone(), "test-secret".to_string()),
        config: AppConfig::test(),
    }
}

/// 测试配置
impl AppConfig {
    pub fn test() -> Self {
        Self {
            database_url: "sqlite::memory:".to_string(),
            jwt_secret: "test-secret".to_string(),
            storage_path: "/tmp/test-storage".to_string(),
            max_file_size: 10 * 1024 * 1024, // 10MB
        }
    }
}

```

2.4 运行单元测试

运行所有测试

```
cargo test
```

运行特定模块测试

```
cargo test case_service
```

```
# 运行测试并显示输出
cargo test -- --nocapture

# 生成覆盖率报告
cargo install cargo-tarpaulin
cargo tarpaulin --out Html

# 只运行快速测试
cargo test --test-threads=4
```

三、集成测试

3.1 API 测试框架

```
// tests/api_tests.rs
use axum::Router;
use http_body_util::BodyExt;
use serde_json::json;
use crate::test_utils::{create_test_app, create_test_user};

/// 测试认证流程
#[tokio::test]
async fn test_auth_flow() {
    let app = create_test_app().await;

    // 1. 注册用户
    let register_response = app
        .post("/api/v1/auth/register")
        .json(&json!({
            "username": "testuser",
            "email": "test@example.com",
            "password": "password123"
        }))
        .send()
        .await
        .unwrap();

    assert_eq!(register_response.status(), 200);
```

```

// 2. 登录获取 Token
let login_response = app
    .post("/api/v1/auth/login")
    .json(&json!({
        "username": "testuser",
        "password": "password123"
    }))
    .send()
    .await
    .unwrap();

assert_eq!(login_response.status(), 200);

let body: serde_json::Value = login_response.json().await.unwrap();
let token = body["data"]["access_token"].as_str().unwrap();

// 3. 使用 Token 访问受保护接口
let me_response = app
    .get("/api/v1/auth/me")
    .header("Authorization", format!("Bearer {}", token))
    .send()
    .await
    .unwrap();

assert_eq!(me_response.status(), 200);
assert_eq!(me_response["data"]["username"], "testuser");
}

/// 测试案件 CRUD
#[tokio::test]
async fn test_case_crud() {
    let app = create_test_app().await;
    let token = create_test_user(&app).await;

    // 1. 创建案件
    let create_response = app
        .post("/api/v1/cases")
        .header("Authorization", format!("Bearer {}", token))
        .json(&json!({
            "name": "测试案件",

```

```

        "description": "用于测试的案件"
    )))
    .send()
    .await
    .unwrap();

assert_eq!(create_response.status(), 200);
let case_id = create_response["data"]["id"].as_str().unwrap();

// 2. 获取案件详情
let get_response = app
    .get(&format!("/api/v1/cases/{}", case_id))
    .header("Authorization", format!("Bearer {}", token))
    .send()
    .await
    .unwrap();

assert_eq!(get_response.status(), 200);
assert_eq!(get_response["data"]["name"], "测试案件");

// 3. 更新案件
let update_response = app
    .put(&format!("/api/v1/cases/{}", case_id))
    .header("Authorization", format!("Bearer {}", token))
    .json(&json!({
        "name": "更新后的案件",
        "description": "更新后的描述"
    })))
    .send()
    .await
    .unwrap();

assert_eq!(update_response.status(), 200);

// 4. 删除案件
let delete_response = app
    .delete(&format!("/api/v1/cases/{}", case_id))
    .header("Authorization", format!("Bearer {}", token))
    .send()
    .await

```



```

        .unwrap();

    assert_eq!(delete_response.status(), 200);

    // 5. 验证案件已删除
    let get_after_delete = app
        .get(&format!("/api/v1/cases/{}", case_id))
        .header("Authorization", format!("Bearer {}", token))
        .send()
        .await
        .unwrap();

    assert_eq!(get_after_delete.status(), 404);
}

```

3.2 文件上传测试

```

// tests/file_upload_test.rs
use axum::body::Body;
use http::{Request, StatusCode};
use mime_guess::mime;

#[tokio::test]
async fn test_file_upload() {
    let app = create_test_app().await;
    let token = create_test_user(&app).await;

    // 创建测试案件
    let case_id = create_test_case(&app, &token).await;

    // 准备测试文件
    let file_content = b"测试文件内容";
    let boundary = "----WebKitFormBoundary";

    let body = format!(
        "--{}\\r\\n\\
        Content-Disposition: form-data; name=\"file\"; filename=\"test.pdf\"
        Content-Type: application/pdf\\r\\n\\r\\n\\
        {}\\r\\n\\
        --{}--\\r\\n",
        boundary,
        file_content.to_string(),
        boundary
    );
}

```

```

        boundary,
        String::from_utf8_lossy(file_content),
        boundary
    );

// 上传文件
let upload_response = app
    .post(&format!("/api/v1/cases/{}/files", case_id))
    .header("Authorization", format!("Bearer {}", token))
    .header("Content-Type", format!("multipart/form-data; boundary={}",
    .body(body)
    .send()
    .await
    .unwrap());

assert_eq!(upload_response.status(), 200);

let evidence_id = upload_response["data"]["id"].as_str().unwrap();
assert!(evidence_id.starts_with("evidence_"));
}

#[tokio::test]
async fn test_file_upload_too_large() {
    let app = create_test_app().await;
    let token = create_test_user(&app).await;
    let case_id = create_test_case(&app, &token).await;

    // 准备超大文件 (超过 100MB)
    let large_content = vec![0u8; 101 * 1024 * 1024];

    let upload_response = app
        .post(&format!("/api/v1/cases/{}/files", case_id))
        .header("Authorization", format!("Bearer {}", token))
        .body(large_content)
        .send()
        .await
        .unwrap();

    assert_eq!(upload_response.status(), StatusCode::BAD_REQUEST);

```

```
    assert_eq!(upload_response["code"], 420102);  
}
```

3.3 运行集成测试

运行所有集成测试

```
cargo test --test '*'
```

运行特定测试文件

```
cargo test --test api_tests
```

运行测试并显示日志

```
RUST_LOG=debug cargo test --test api_tests -- --nocapture
```

四、E2E 测试

4.1 Playwright 配置

```
// e2e/playwright.config.ts  
import { defineConfig } from '@playwright/test';  
  
export default defineConfig({  
  testDir: './tests',  
  timeout: 30000,  
  expect: {  
    timeout: 5000,  
  },  
  use: {  
    baseURL: 'http://localhost:3000',  
    screenshot: 'only-on-failure',  
    video: 'retain-on-failure',  
  },  
  projects: [  
    {  
      name: 'chromium',  
      use: { browserName: 'chromium' },  
    },  
    {  
      name: 'firefox',  
      use: { browserName: 'firefox' },  
    },  
  ],  
});
```

```
    },  
  ],  
  reporter: [['html'], ['list']],  
});
```

4.2 E2E 测试用例

```
// e2e/tests/login.spec.ts  
import { test, expect } from '@playwright/test';  
  
test.describe('登录流程', () => {  
  test('成功登录', async ({ page }) => {  
    // 访问登录页  
    await page.goto('/login');  
  
    // 填写表单  
    await page.fill('input[placeholder="用户名"]', 'testuser');  
    await page.fill('input[type="password"]', 'password123');  
  
    // 提交登录  
    await page.click('button[type="submit"]');  
  
    // 验证跳转到首页  
    await expect(page).toHaveURL('/');  
    await expect(page.locator('text=欢迎回来')).toBeVisible();  
  });  
  
  test('登录失败 - 密码错误', async ({ page }) => {  
    await page.goto('/login');  
  
    await page.fill('input[placeholder="用户名"]', 'testuser');  
    await page.fill('input[type="password"]', 'wrongpassword');  
  
    await page.click('button[type="submit"]');  
  
    // 验证错误提示  
    await expect(page.locator('text=用户名或密码错误')).toBeVisible();  
  });  
  
  test('登录后 Token 持久化', async ({ page }) => {  
    // 登录
```

```
    await page.goto('/login');
    await page.fill('input[placeholder="用户名"]', 'testuser');
    await page.fill('input[type="password"]', 'password123');
    await page.click('button[type="submit"]');

    // 验证 localStorage 中有 Token
    const token = await page.evaluate(() => localStorage.getItem('access_token'));
    expect(token).toBeTruthy();

    // 刷新页面后仍然登录
    await page.reload();
    await expect(page).toHaveURL('/');
  });
});

// e2e/tests/case.spec.ts
test.describe('案件管理', () => {
  test.beforeEach(async ({ page }) => {
    // 前置登录
    await page.goto('/login');
    await page.fill('input[placeholder="用户名"]', 'testuser');
    await page.fill('input[type="password"]', 'password123');
    await page.click('button[type="submit"]');
  });

  test('创建案件', async ({ page }) => {
    await page.goto('/cases');

    // 点击创建按钮
    await page.click('text=新建案件');

    // 填写表单
    await page.fill('input[placeholder="案件名称"]', 'E2E 测试案件');
    await page.fill('textarea[placeholder="案件描述"]', '用于 E2E 测试');

    // 提交
    await page.click('button[type="submit"]');

    // 验证创建成功
    await expect(page.locator('text=创建成功')).toBeVisible();
    await expect(page.locator('text=E2E 测试案件')).toBeVisible();
  });
});
```

```
});

test('上传证据文件', async ({ page }) => {
  // 进入案件详情
  await page.click('text=测试案件');

  // 上传文件
  const fileInput = page.locator('input[type="file"]');
  await fileInput.setInputFiles('e2e/fixtures/test.pdf');

  // 验证上传成功
  await expect(page.locator('text=上传成功')).toBeVisible();
  await expect(page.locator('text=test.pdf')).toBeVisible();
});

test('创建时间轴节点', async ({ page }) => {
  await page.goto('/timeline');

  // 点击创建节点
  await page.click('text=新建节点');

  // 填写节点信息
  await page.fill('input[placeholder="节点标题"]', '合同签订');
  await page.fill('input[type="date"]', '2024-01-15');
  await page.fill('textarea[placeholder="节点描述"]', '张三与李四签订合同');

  // 提交
  await page.click('button[type="submit"]');

  // 验证节点出现在时间轴上
  await expect(page.locator('text=合同签订')).toBeVisible();
});
});
```

4.3 运行 E2E 测试

```
# 安装依赖
cd e2e
npm install

# 运行所有测试
```

```
npx playwright test

# 运行特定测试
npx playwright test login.spec.ts

# 有头模式运行（显示浏览器）
npx playwright test --headed

# 生成报告
npx playwright test --reporter=html
npx playwright show-report
```

五、性能测试

5.1 wrk 基准测试

```
# 安装 wrk
brew install wrk # macOS
apt-get install wrk # Linux

# 测试登录接口
wrk -t4 -c100 -d30s \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer {token}" \
  http://localhost:8000/api/v1/cases

# 测试结果示例：
# Running 30s test @ http://localhost:8000/api/v1/cases
# 4 threads and 100 connections
# Thread Stats Avg Stdev Max +/- Stdev
# Latency 45.23ms 12.34ms 156.78ms 85.23%
# Req/Sec 556.78 45.67 678.00 82.34%
# 16543 requests in 30s, 15.23MB read
# Requests/sec: 551.43
# Transfer/sec: 519.45KB
```

5.2 locust 负载测试

```
# tests/performance/locustfile.py
from locust import HttpUser, task, between
import random

class LegalMindsUser(HttpUser):
    wait_time = between(1, 3)

    def on_start(self):
        # 登录获取 Token
        response = self.client.post("/api/v1/auth/login", json={
            "username": "testuser",
            "password": "password123"
        })
        self.token = response.json()["data"]["access_token"]
        self.headers = {"Authorization": f"Bearer {self.token}"}

    @task(3)
    def get_cases(self):
        self.client.get("/api/v1/cases", headers=self.headers)

    @task(2)
    def get_timeline(self):
        case_id = random.choice(["case_001", "case_002", "case_003"])
        self.client.get(
            f"/api/v1/cases/{case_id}/timeline/nodes",
            headers=self.headers
        )

    @task(1)
    def search(self):
        keywords = ["合同", "证据", "张三", "李四"]
        keyword = random.choice(keywords)
        self.client.get(
            f"/api/v1/search?keyword={keyword}",
            headers=self.headers
        )
```



```
# 运行 locust
locust -f tests/performance/locustfile.py --host=http://localhost:8000

# 访问 Web UI
# http://localhost:8089
```

5.3 性能指标要求

接口	QPS 目标	P95 延迟	P99 延迟
GET /cases	>500	<100ms	<200ms
GET /timeline/nodes	>300	<200ms	<500ms
POST /files/upload	>50	<1s	<2s
GET /search	>200	<300ms	<500ms
POST /auth/login	>100	<200ms	<500ms

六、安全测试

6.1 代码安全审计

```
# 安装 cargo-audit
cargo install cargo-audit

# 运行安全审计
cargo audit

# 定期审计 (CI/CD 中)
cargo audit --deny warnings
```

6.2 依赖漏洞检查

```
# 检查 Rust 依赖
cargo outdated
cargo audit

# 检查 Python 依赖
pip install safety
safety check -r python/requirements.txt

# 检查 Node.js 依赖
```

```
npm audit
npm audit fix
```

6.3 OWASP 测试项

测试项	测试方法	工具
SQL 注入	尝试注入特殊字符	sqlmap
XSS	尝试注入	